Warehouse Inventory Management System

CS6103 — Introduction to Java, Spring 2026

Student	Bhavith Chandra
Net ID	bc4066
Course	CS6103 — Introduction to Java
Semester	Spring 2026

Abstract

This report presents the design and implementation of a multi-user Warehouse Inventory Management System developed as a final project for CS6103. The system follows a three-tier client-server architecture built entirely in Java, using TCP sockets for networking, JavaFX 21 for the graphical user interface, and SQLite for persistent storage. A central focus of the implementation is correctness under concurrency: the system guarantees that two clients attempting to deduct the last unit of a product simultaneously will never produce a double-deduction, a property proven by an automated stress test included with the submission.

1. Introduction

Modern warehouses require real-time, multi-user access to inventory data. Staff need to log stock movements instantly, managers need accurate stock levels at a glance, and the system must remain consistent even when multiple users act simultaneously. This project builds a fully functional desktop application that satisfies all of these requirements using core Java technologies covered in CS6103.

The system supports:

- Real-time product catalogue management (create, read, update, delete)
- Stock-level tracking with add, remove, and absolute-adjust operations
- Automatic low-stock alerting with inline restock shortcuts
- A full, immutable audit transaction log
- Role-based access control (Admin vs Staff)
- Live push-event updates — all connected clients refresh automatically
- CSV export of inventory and transaction history

2. System Architecture

The application follows a strict three-tier architecture that cleanly separates presentation, business logic, and data concerns.

Tier 1 — Presentation (JavaFX Client)

The client is a JavaFX 21 desktop application (**InventoryClient.java**). It contains no business logic; every user action becomes a serialised *Request* sent over a TCP socket. The UI runs entirely on the JavaFX Application Thread; all network I/O runs on background threads, and results are marshalled back to the UI via `Platform.runLater()`.

Tier 2 — Application Server (Java TCP Server)

InventoryServer opens a `ServerSocket` on port 5555 and spawns one `Thread(ClientHandler)` per accepted connection. **ClientHandler** reads *Request* objects, routes them to **InventoryService**, and writes *Response* objects back. When a mutation succeeds the server broadcasts a push event to all subscribed clients so their UIs refresh without polling.

Tier 3 — Data (SQLite via JDBC)

DatabaseManager owns a single JDBC `Connection` to an SQLite file (`data/warehouse.db`). WAL journal mode is enabled so readers never block writers. The schema has three tables: *users*, *products*, and *transactions*.

Network Protocol

All messages are Java-serialised objects. *Request* carries a `Type` enum (14 verbs: LOGIN, LIST_PRODUCTS, ADD_STOCK, REMOVE_STOCK, ...) plus a parameter map and a `requestId = System.nanoTime()`. *Response* carries a `Status` (OK / ERROR / EVENT), a payload, and the matching `requestId`. The client's **ServerConnection** correlates incoming responses to outstanding `CompletableFutures` by that id, enabling fully asynchronous networking on a single socket.

3. Concurrency Design

The headline correctness property from the proposal is: *"if two users try to deduct the last unit of a product at the same time, only one should succeed."* The system enforces this with a two-layer defence inside `InventoryService.adjustStock()`.

Layer 1 — Per-product ReentrantLock

A `ConcurrentHashMap` maps each product ID to a `ReentrantLock`. Every stock mutation acquires the lock for that product before doing any I/O. Writes to *different* products proceed in parallel; writes to the *same* product are strictly serialised.

Layer 2 — JDBC Transaction

Inside the lock, the code calls `setAutoCommit(false)`, re-reads the current quantity from the database within the same transaction (preventing stale cached reads), validates the business rule (`quantity + delta ≥ 0`), applies the UPDATE, writes the audit log row, and commits. On any exception `safeRollback()` is called. The database schema also enforces `CHECK (quantity ≥ 0)` as a final safety net.

Why two layers? The database constraint alone would let both clients attempt the deduction and give one a confusing SQLException. The application lock ensures one client gets a clean "out of stock" message and the other a successful response, which is the expected user experience.

4. Authentication and Security

Passwords are stored as salted SHA-256 hashes (`base64(salt):base64(sha256(salt||password))`) using the **PasswordHasher** utility class. Constant-time comparison is used during verification to prevent timing attacks.

Two roles are supported:

- **ADMIN** — full access: create products, update products, delete products, all stock operations
- **STAFF** — restricted: stock operations only (add / remove / adjust), read-only catalogue views

The server enforces login-first: any request other than LOGIN before authentication is rejected with "Not authenticated." Role checks are enforced in `ClientHandler.requireRole()` on every

admin-only operation.

Username	Password	Role
`admin`	`admin123`	`ADMIN`
`staff`	`staff123`	`STAFF`

Default seeded credentials (first run only).

5. Key Implementation Files

Server

InventoryServer.java	ServerSocket accept loop, handler registry, broadcast engine
ClientHandler.java	Per-connection request dispatch; writeLock prevents interleaved writes
InventoryService.java	All business logic, concurrency guards, JDBC transactions
DatabaseManager.java	Schema creation, WAL / FK PRAGMAs, single JDBC connection lifetime

Client

InventoryClient.java	JavaFX Application; sidebar navigation; Ctrl+R/N/F/E shortcuts
ServerConnection.java	Async socket I/O; CompletableFuture correlation; push-event dispatch
DashboardView.java	KPI cards, PieChart by category, BarChart top-10, activity feed
InventoryView.java	Searchable / filterable TableView; Add Stock / Remove Stock dialogs
StockOpsView.java	Dedicated receive / ship / cycle-count panel with live preview
LowStockView.java	Auto-filtered low-stock list with one-click Restock shortcut
HistoryView.java	Transaction audit log with colour-coded action badges; CSV export

6. Testing and Verification

ProposalVerify.java — End-to-end verification

Connects to a live server and exercises every proposal requirement across 29 automated checks: all 6 CRUD operations, all 3 stock mutation types, over-deduction guard, low-stock report, full transaction log inspection, role-based access control, unauthenticated request rejection, and push-event delivery.

```
Result: 29 / 29 PASS
```

ConcurrencyStressTest.java — Race-condition proof

Resets a product to N units, launches T client threads each repeatedly calling REMOVE_STOCK(1) until rejected, then reads the final stock.

```
Sample run (N=50, T=15)
successful deductions: 50
insufficient rejects: 15
other errors: 0
final stock: 0
Result: PASS — no double-deduction
```

7. How to Run

Prerequisites

- JDK 17 or later (tested on OpenJDK 25)
- JavaFX SDK 21.0.5 — download from **https://gluonhq.com/products/javafx/** and place the extracted folder at `lib/javafx-sdk-21.0.5/`
- sqlite-jdbc-3.46.1.3.jar, slf4j-api-2.0.13.jar, slf4j-simple-2.0.13.jar are already included in `lib/`

Step-by-step

Step 1 — Compile

`bash scripts/compile.sh`

Step 2 — Start the server (Terminal 1)

`bash scripts/run-server.sh`

On first run the server creates `data/warehouse.db`, seeds 2 users and 10 sample products, and prints:

`Warehouse server listening on port 5555`

Step 3 — Start the client (Terminal 2)

`bash scripts/run-client.sh`

A connection dialog appears. Defaults: localhost:5555 / admin / admin123. Open additional terminals for a multi-user demo.

Step 4 — Stress test (optional)

`bash scripts/run-stress.sh`

Or with custom parameters: `bash scripts/run-stress.sh [productId] [startStock] [threads]`

Eclipse Import

- File → Import → General → Existing Projects into Workspace → select this folder → Finish
- The included `.project` and `.classpath` configure all JARs automatically
- Client run configuration VM args: `--module-path lib/javafx-sdk-21.0.5/lib --add-modules javafx.controls,javafx.fxml,javafx.graphics,javafx.base`

UI Navigation and Keyboard Shortcuts

View	Description	Shortcut
Dashboard	KPI cards, charts, activity feed	
Inventory	Product table — search, filter, CRUD, stock dialogs	Ctrl+F / Ctrl+N
Stock Operations	Receive / ship / cycle-count panel	
Low Stock Alerts	Products at or below reorder level	
Transaction History	Full audit log with colour-coded badges	
Refresh all	Pull latest data from server	Ctrl+R
Export CSV	Save inventory to CSV file	Ctrl+E

8. Conclusion

The Warehouse Inventory Management System demonstrates the application of core Java concepts — TCP sockets, multithreading, JavaFX, and JDBC — in a cohesive, production-quality application. The two-layer concurrency strategy (per-product `ReentrantLock` + JDBC transaction) correctly solves the classic "last unit" race condition identified in the project proposal as the central correctness challenge. All 29 automated proposal-verification checks pass, and the concurrency stress test proves no double-deduction under heavy parallel load.

Source code and full documentation available at:
https://github.com/Bhavith-Chandra/warehouse-inventory-management